ITA0615 – MACHINE LEARNING
C03 AT1

By:
Harshith G
192525394

Annexure A

Experiments

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Answer:

Aim: To implement a Naïve Bayes classifier on a real-world dataset, compare different feature sets, study the effect of the feature-independence assumption, and compare results with and without Laplace smoothing.

Dataset and tools: Breast Cancer Wisconsin (Diagnostic) dataset – 569 samples, 30 numeric features, 2 classes (malignant / benign). 70 % training (398 samples) and 30 % testing (171 samples), stratified, random_state = 42. Python with scikit-learn. Each feature was discretised into 10 quantile bins so that categorical Naïve Bayes (which uses counts, and hence needs smoothing) could be applied.

Principle:
Naïve Bayes applies Bayes theorem with the assumption that features are conditionally independent given the class: $P(C \mid x1 \ldots xn) \propto P(C) \cdot \prod P(xi \mid C)$. The class with the highest posterior is predicted. With Laplace smoothing, $P(xi = v \mid C) = (\text{count}(v, C) + \alpha) / (\text{count}(C) + \alpha \cdot k)$, where k is the number of bins and $\alpha = 1$.

Procedure:
1. Load the dataset and split it into training and test sets.
2. Discretise every feature into 10 quantile bins (fitted on the training data only).
3. Select four feature sets: A – first 5 features; B – first 10 features; C – all 30 features; D – 10 highly correlated size features (radius, perimeter, area at mean / worst / error level and related).
4. Train CategoricalNB twice on each set: once with $\alpha \approx 0$ (no smoothing) and once with $\alpha = 1$ (Laplace).
5. Predict on the test set and compute accuracy, precision and recall.
6. Check the correlation between the features of Set D and count the zero-count cells in the model.

Program (key part):

```
kb = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='quantile')
Btr = kb.fit_transform(Xtr).astype(int); Bte = kb.transform(Xte).astype(int)
def run(cols, alpha):
    m = CategoricalNB(alpha=alpha, min_categories=10).fit(Btr[:, cols], ytr)
    p = m.predict(Bte[:, cols])
    return accuracy_score(yte,p), precision_score(yte,p), recall_score(yte,p)
for name, cols in feature_sets.items():
    print(name, run(cols, 1e-10), run(cols, 1.0))   # no smoothing vs Laplace
```

Observations (test set):

Feature set	No smoothing Acc	Prec	Rec	Laplace (α=1) Acc	Prec	Rec
A – 5 features	0.912	0.918	0.944	0.912	0.918	0.944
B – 10 features	0.930	0.952	0.935	0.930	0.952	0.935
C – all 30 features	0.947	0.954	0.963	0.942	0.962	0.944
D – 10 correlated features	0.947	0.945	0.972	0.930	0.936	0.953
Gaussian NB (30 features)	0.947	0.938	0.981	–	–	–

Analysis:

Different feature sets: Accuracy rose from 91.2 % (5 features) to 93.0 % (10 features) and 94.7 % (all 30 features). More features give more evidence, so the class posterior becomes better separated, and recall for malignant/benign cases stays above 93 %.

Effect of the independence assumption: The size-related features in Set D are strongly correlated (pairwise correlation ≥ 0.94), so the independence assumption is clearly violated. Naïve Bayes then counts the same evidence several times and its posterior probabilities become over-confident (close to 0 or 1). Even so, accuracy stayed at 94.7 %, because classification needs only the correct ranking of the two posteriors, not their exact values. Adding redundant features therefore did not give any extra benefit over the 30-feature model, and a correlation-aware model (e.g. feature selection or a Bayesian network) would be the better choice if calibrated probabilities are needed.

Effect of Laplace smoothing: The model with all 30 features contained 37 zero-count cells, i.e. bin values never observed for one of the classes. Without smoothing any such value in a test sample forces that class probability to zero. With $\alpha = 1$ these zeros are replaced by small non-zero estimates. In this experiment, however, smoothing made no difference for Sets A and B and lowered accuracy slightly for Sets C and D ($0.947 \rightarrow 0.942$ and $0.947 \rightarrow 0.930$), because the training set is reasonably large and adding a pseudo-count shifts the estimates of the sparse bins towards uniform and slightly reduces discrimination. Smoothing is most valuable when data are sparse (e.g. text classification with a large vocabulary), where one unseen word would otherwise zero out a document.

Conclusion:

Naïve Bayes works well on this dataset ($\approx$ 95 % accuracy) in spite of violated independence. Probability estimation matters: unsmoothed estimates are risky when counts are small, and Laplace smoothing is a safe default that avoids zero probabilities at a small cost in accuracy when data are plentiful.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Answer:

Aim: To estimate the mean and variance of a Gaussian distribution by MLE and by Bayesian estimation, and to study the influence of the prior and of the sample size.

Dataset and tools: Samples drawn from a Gaussian distribution with true mean $\mu = 50$ and true variance $\sigma^2 = 100$ (NumPy, seed 7), using sample sizes n = 5, 10, 30, 100 and 1000. Using synthetic data lets us compare every estimate with the known true value.

Principle:
MLE chooses the parameters that maximise the likelihood of the data: $\mu_MLE = (1/n)\ \Sigma\ xi$ and $\sigma^2_MLE = (1/n)\ \Sigma\ (xi - \mu_MLE)^2$. The variance estimate is biased (it divides by n). Bayesian estimation combines the likelihood with a prior. A Normal–Inverse-Gamma prior is conjugate for a Gaussian with unknown mean and variance, so the posterior is available in closed form:

$\kappa n = \kappa 0 + n;\quad \mu n = (\kappa 0\ \mu 0 + n\ \bar{x}) / \kappa n;\quad \alpha n = \alpha 0 + n/2;\quad \beta n = \beta 0 + \tfrac{1}{2}\ \Sigma(xi - \bar{x})^2 + \kappa 0\ n\ (\bar{x} - \mu 0)^2 / (2\ \kappa n)$

Posterior mean of $\mu = \mu n$ and posterior mean of $\sigma^2 = \beta n / (\alpha n - 1)$. The prior used was deliberately slightly wrong: $\mu 0 = 40$, $\kappa 0 = 5$

Procedure:
1. Generate n samples from $N(50, 10^2)$ for each sample size.
2. Compute the MLE of the mean and variance.
3. Compute the posterior parameters and the Bayesian estimates of mean and variance.
4. Repeat the experiment 2000 times for n = 5, 30 and 1000 and record the average absolute error of each method.
5. Compare the estimates with the true values and with each other.

Program (key part):

```
x = rng.normal(50, 10, n); xb = x.mean(); s2 = x.var()        # MLE
kn = k0 + n; mun = (k0*mu0 + n*xb) / kn; an = a0 + n/2
bn = b0 + 0.5*((x-xb)**2).sum() + k0*n*(xb-mu0)**2/(2*kn)
bayes_mean = mun; bayes_var = bn / (an - 1)
```

Observations (one sample per n; true $\mu = 50$, $\sigma^2 = 100$):

n	MLE mean	MLE variance	Bayes mean	Bayes variance	Posterior SD of μ
5	47.36	16.32	43.68	57.45	2.40
10	49.29	46.05	46.19	74.86	2.23
30	45.54	80.30	44.75	83.54	1.54
100	49.10	80.62	48.67	84.20	0.90
1000	49.39	92.68	49.34	93.04	0.30

Average absolute error over 2000 repetitions:

n	Mean error – MLE	Mean error – Bayes	Variance error – MLE	Variance error – Bayes
5	3.60	5.02	49.3	31.9
30	1.49	1.76	20.7	18.4
1000	0.25	0.25	3.5	3.5

Analysis:

Small samples: With n = 5 the MLE variance (16.32 in this sample) is badly underestimated and is biased low, and its mean error is large. The Bayesian variance (57.45) is pulled towards the prior and is much closer to the truth: the average variance error drops from 49.3 to 31.9. The Bayesian mean, however, is pulled towards the wrong prior mean of 40 and is worse (error 5.02 vs 3.60). A prior therefore helps when it is reasonable and hurts when it is wrong.

Large samples: As n grows, the weight of the prior ($\kappa_0 = 5$ pseudo-observations) becomes negligible compared with the data. At n = 1000 both methods give almost identical estimates (49.39 vs 49.34 for the mean) with identical errors, and both converge to the true values. The posterior SD of μ shrinks as $\sigma/\sqrt{n}$ ($2.40 \rightarrow 0.30$), which equals the MLE standard error.

Reliability: MLE is simple, unbiased for the mean and consistent, but unreliable with very few samples, whereas Bayesian estimation gives a full posterior distribution, so the uncertainty of the parameters is expressed explicitly and extreme estimates are avoided. Its reliability depends on a sensible prior.

Conclusion: MLE and Bayesian estimation agree for large samples; for small samples the prior regularises the estimate and reduces variance error, at the cost of bias if the prior is wrong.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

Answer:

Aim: To implement the EM algorithm for a Gaussian Mixture Model (GMM) and to study how initialisation and the number of clusters K affect convergence speed, final log-likelihood and local optima.

Dataset and tools: Synthetic 2-D data with 440 points from three Gaussian components: 200 points around (0, 0) with $\sigma = 1$, 200 points around (3, 3) with $\sigma = 1$ (these two overlap) and 40 points around (8, 0) with $\sigma = 0.6$. Python (NumPy, SciPy, scikit-learn for k-means initialisation). EM stops when the change in log-likelihood is below 10^{-6} or after 300 iterations.

Algorithm:

1. Initialise the means μk, covariances Σk and weights πk (random data points, or k-means centres).
2. E-step: compute the responsibilities $\gamma ik = \pi k \, N(xi \mid \mu k, \Sigma k) / \Sigma j \, \pi j \, N(xi \mid \mu j, \Sigma j)$.
3. M-step: $Nk = \Sigma i \, \gamma ik$; $\mu k = \Sigma i \, \gamma ik \, xi / Nk$; $\Sigma k = \Sigma i \, \gamma ik \, (xi - \mu k)(xi - \mu k)^T / Nk$; $\pi k = Nk / N$.
4. Compute the log-likelihood $\Sigma i \ln \Sigma k \, \pi k \, N(xi \mid \mu k, \Sigma k)$.
5. Repeat steps 2–4 until the log-likelihood converges (it never decreases between iterations).

Program (key part):

```
for it in range(1, maxit+1):
    p  = np.column_stack([pi[k]*mvn.pdf(X, mu[k], cov[k]) for k in range(K)])
    ll = np.log(p.sum(1)).sum(); g = p / p.sum(1, keepdims=True)      # E-step
    Nk = g.sum(0); pi = Nk/n; mu = (g.T @ X) / Nk[:, None]            # M-step
    for k in range(K):
        D = X - mu[k]; cov[k] = (g[:, k, None]*D).T @ D / Nk[k] + 1e-6*np.eye(d)
    if abs(ll - prev) < tol: break
    prev = ll
```

Observations (30 runs for each setting):

Initialisation	K	Avg. iterations	Avg. final log-likelihood	Worst / best log-likelihood
Random points	2	39.7	−1692.7	−1768.0 / −1673.3
k-means	2	68.6	−1673.3	−1673.3 / −1673.3
Random points	3	32.0	−1593.0	−1593.0 / −1593.0
k-means	3	22.0	−1593.0	−1593.0 / −1593.0
Random points	4	250.3	−1588.7	−1590.9 / −1580.9
k-means	4	249.1	−1588.7	−1591.1 / −1587.8

Local optima (100 runs, K = 2): 23 random initialisations ended in a poorer solution (log-likelihood down to −1768.0 against the best −1673.3), whereas 0 of 100 k-means initialisations did. For K = 3, all 100 runs of both types reached the same optimum (−1593.0).

Analysis:

Number of clusters: K = 3 (the true number) gave the fastest and most stable convergence (22–32 iterations, identical solution in every run). K = 2 forces two components to cover three groups, so the likelihood is much lower and the solution depends on the start. K = 4 improved the log-likelihood by only about 4 units (−1593.0 → −1588.7) while adding 6 parameters; the BIC penalty ($6 \cdot \ln 440 \approx 36$) far outweighs this gain, so K = 3 is selected. At K = 4 the extra component slowly splits a real cluster, so EM crawls (≈ 250 iterations, close to the 300 cap).

Initialisation: Random initialisation can place two initial means in the same cluster and none in another; EM then converges to a local maximum with a lower likelihood (23 % of the K = 2 runs). k-means initialisation starts near good centres, giving the same optimum every time in these tests, though in the K = 2 case it needed more iterations on average (68.6 vs 39.7) because its solution is the better, fully converged one.

Convergence: The log-likelihood increased monotonically in every run, as EM guarantees, but only to a local – not necessarily global – maximum.

Strategies to improve stability and performance:

1. Initialise with k-means or k-means++ rather than random points.
2. Run EM several times from different starts and keep the solution with the highest log-likelihood.
3. Add a small regularisation term to the covariances (1e-6·I) to avoid singular components that collapse onto a single point.
4. Choose K using BIC / AIC or cross-validated likelihood instead of the likelihood alone.
5. Use a looser tolerance or a maximum iteration limit, or deterministic-annealing / split-and-merge variants, to avoid slow convergence.

Conclusion: EM reliably improves the likelihood at every step, but its final result depends on initialisation and K. Good initialisation, multiple restarts and model-selection criteria give stable, near-optimal mixtures.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Answer:

Aim: To construct a Bayesian Belief Network (BBN) for a home-alarm problem, perform inference under different evidence, and analyse the effect of changing conditional probabilities.

Problem domain: A burglar alarm (A) can be triggered by a burglary (B) or an earthquake (E). Two neighbours, John (J) and Mary (M), may phone when they hear the alarm. All variables are Boolean. The network structure is B → A ← E, A → J, A → M, so the joint distribution factorises as P(B,E,A,J,M) = P(B) P(E) P(A | B,E) P(J | A) P(M | A).

Conditional probability tables:

Node	Probability values
P(B), P(E)	P(B) = 0.001, P(E) = 0.002
P(A | B, E)	B,E = T,T: 0.95; T,F: 0.94; F,T: 0.29; F,F: 0.001
P(J | A)	A = T: 0.90; A = F: 0.05
P(M | A)	A = T: 0.70; A = F: 0.01

Procedure:

1. Define the network and the CPTs above.
2. Perform exact inference by enumeration: sum the joint probability over all hidden variables that agree with the evidence and normalise.
3. Query the posterior probability of Burglary, Earthquake and Alarm under different evidence sets.
4. Modify individual CPT entries one at a time and recompute P(B | J = T, M = T) to test the sensitivity of the model.

Program (key part):

```
def query(P, q, ev):                # exact inference by enumeration
   num = den = 0
   for b,e,a,j,m in itertools.product([0,1], repeat=5):
      v = dict(B=b,E=e,A=a,J=j,M=m)
      if any(v[k] != x for k, x in ev.items()): continue
      p = joint(b,e,a,j,m,P); den += p
      if v[q] == 1: num += p
   return num / den
```

Observations 1 – posteriors for different evidence:

Evidence	P(Burglary)	P(Earthquake)	P(Alarm)
None (prior)	0.0010	0.0020	0.0025
J = T	0.0163	0.0114	0.0434

Evidence	P(Burglary)	P(Earthquake)	P(Alarm)
J = T, M = T	0.2842	0.1761	0.7607
J = T, M = T, E = T	0.0033	1.0000	0.9981
A = T	0.3736	0.2310	1.0000

Observations 2 – sensitivity of P(B | J = T, M = T) to CPT changes (base value 0.2842):

Modified CPT entry	New P(B | J, M)
P(B) 0.001 → 0.01	0.8002
P(A | ¬B, ¬E) 0.001 → 0.05	0.0180
P(J | A) 0.90 → 0.60	0.2538
P(M | A) 0.70 → 0.40	0.2409

Analysis:

Evidence propagation: A single call from John raises the belief in a burglary from 0.1 % to 1.6 %, and calls from both John and Mary raise it to 28.4 %, because two independent witnesses make the alarm (76 %) much more likely, and the alarm is a direct effect of a burglary.

Dependencies / explaining away: Burglary and Earthquake are independent a priori but become dependent once their common effect (the alarm) is observed. When we also learn that an earthquake occurred, P(Burglary) drops from 0.2842 to 0.0033: the earthquake "explains away" the alarm. This reasoning cannot be captured by a model that ignores the network structure.

Robustness: The posterior is very sensitive to the prior P(B) (0.001 → 0.01 raises the posterior from 0.28 to 0.80) and to the false-alarm rate P(A | ¬B, ¬E) (0.001 → 0.05 lowers it from 0.28 to 0.02), but only mildly sensitive to the reliability of the neighbours, P(J | A) and P(M | A) (0.28 → 0.25 and 0.24). Hence the quality of the root priors and of the alarm CPT must be estimated carefully, while the neighbour CPTs may be approximate.

Handling uncertainty: Every variable is represented by a probability distribution, the network factorises the joint distribution into small local tables (here 10 numbers instead of 31), and evidence updates beliefs coherently through Bayes rule. The output is a degree of belief rather than a hard yes/no decision.

Conclusion: The Bayesian Belief Network represents dependencies compactly, supports inference in both causal and diagnostic directions, and quantifies how conclusions change with evidence and parameter values.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Answer:

Aim: To implement the Candidate Elimination algorithm, and to study how the size of the hypothesis space (controlled by the number of attributes) affects the number of training examples needed, using the PAC / mistake-bound viewpoint.

Concept and hypothesis space: Conjunctive concepts over n Boolean attributes. Each attribute in a hypothesis can be 0, 1 or "?" (any value), plus the empty hypothesis, so $|H| = 3^n + 1$. For the standard EnjoySport task (6 attributes with 3,2,2,2,2,2 values) $|H| = 4 \cdot 3^5 = 972$ syntactically and $1 + 3 \cdot 2^5 = 97$ semantically distinct hypotheses.

Algorithm (Candidate Elimination):

1. Initialise the specific boundary S to the most specific hypothesis and the general boundary G to the most general one (all "?").
2. For a positive example: remove from G every hypothesis that does not cover it; minimally generalise every member of S that does not cover it.
3. For a negative example: remove from S every hypothesis that covers it; minimally specialise every member of G that covers it, keeping only specialisations that are more general than some member of S.
4. Repeat for every example. The version space is the set of hypotheses between S and G; learning has converged when S = G.

Program (key part):

```
def ce(n, target, examples):
    S = None; G = {tuple('?'*n)}
    for x in examples:
        if covers(target, x):                    # positive example
            G = {g for g in G if covers(g, x)}
            S = list(x) if S is None else [a if a==b else '?' for a,b in zip(S,x)]
        else:                                    # negative example: specialise G
            newG = set()
            for g in G:
                if not covers(g, x): newG.add(g); continue
                for i in range(n):
                    if g[i]=='?' and (S is None or (S[i] in '01' and S[i]!=x[i])):
                        h = list(g); h[i] = '1' if x[i]=='0' else '0'
                        newG.add(tuple(h))
            G = newG
        if S is not None and G == {tuple(S)}: return steps   # converged: S = G
```

Experiment:

For n = 3…8 attributes, 100 random target conjunctions were generated and learned from random labelled examples; the number of examples needed until S = G was recorded and averaged (for n = 8, 90 of the 100 runs converged within 600 examples and only these are averaged). The theoretical bounds are shown for comparison.

Observations:

Attributes n	$\|H\| = 3^n+1$	Avg. examples to converge	Find-S mistake bound (n+1)	Halving mistake bound $\log_2\|H\|$	PAC bound m (ε = 0.1, δ = 0.05)
3	28	12.5	4	4.8	63
4	82	21.3	5	6.4	74
5	244	33.7	6	7.9	85
6	730	72.3	7	9.5	96
7	2188	126.0	8	11.1	107
8	6562	123.8*	9	12.7	118

* average over the 90 runs that converged within 600 examples.

Analysis:

Hypothesis-space size and sample complexity: As the number of attributes grows, |H| increases exponentially (28 → 6562), and the number of examples needed for the version space to collapse to the target grows from about 12 to over 120. Larger spaces keep many consistent hypotheses alive, so more (and more informative, especially negative) examples are needed to eliminate them. More training examples shrink the version space monotonically.

Finite hypothesis spaces: For a finite H, a consistent learner needs only $m \geq (1/\varepsilon)(\ln|H| + \ln(1/\delta))$ examples to reach error $\leq \varepsilon$ with probability $\geq 1 - \delta$. This grows only logarithmically in |H|, i.e. linearly in n for conjunctions (63 → 118 examples), so the learner generalises efficiently. The bound is a worst-case guarantee; the experiment uses random examples, which are less informative than well-chosen ones.

Infinite or unbiased hypothesis spaces: If H contains every possible concept (2^(2^n) concepts), or is infinite (e.g. intervals or hyperplanes over real values), ln|H| is infinite or enormous and the above bound is useless; the VC dimension must be used instead, with $m \geq (1/\varepsilon)(4 \log_2(2/\delta) + 8 \cdot VC(H) \cdot \log_2(13/\varepsilon))$. An unbiased learner can only memorise: S becomes the disjunction of the positive examples and G the negation of the negative ones, so it cannot classify any unseen instance. Generalisation therefore requires an inductive bias such as restricting H to conjunctions.

Mistake-bound interpretation: The mistake-bound model counts the errors made while learning online. Find-S makes at most n + 1 mistakes on conjunctions of n literals (one for the first positive example and at most one for each attribute generalised to "?"), and the Halving algorithm makes at most $\log_2|H|$ mistakes (12.7 for n = 8), because every mistake removes at least half of the version space. These bounds rise slowly with n, whereas the number of random examples needed for exact convergence rises faster, showing that the number of mistakes is far smaller than the number of examples needed to identify the concept exactly.

Conclusion: Candidate Elimination converges to the target concept as examples are added, but the effort grows with the size of the hypothesis space. A restricted (biased) finite hypothesis space gives low sample complexity and bounded mistakes, whereas unrestricted or infinite spaces need VC-dimension arguments and cannot generalise without bias.